UNIDAD N°5: Arquitecturas de Software

Índice

1. **Introducción**
2. **Arquitectura Monolítica**
    - 2.1 Ventajas y Desventajas
    - 2.2 Ejemplos de uso
3. **Arquitectura de Capas**
    - 3.1 Componentes de la Arquitectura de Capas
    - 3.2 Beneficios y limitaciones
    - 3.3 Casos de estudio
4. **Arquitectura Hexagonal (Ports and Adapters)**
    - 4.1 Estructura de una Arquitectura Hexagonal
    - 4.2 Patrón de diseño aplicable
    - 4.3 Casos de uso y ejemplos prácticos
5. **Arquitectura Basada en Servicios (SOA)**
    - 5.1 Características principales
    - 5.2 Comparativa entre SOA y Microservicios
    - 5.3 Escenarios de aplicación
6. **Microservicios**
    - 6.1 Arquitectura de Microservicios: Independencia y modularidad
    - 6.2 Desafíos y patrones comunes
    - 6.3 Herramientas populares y ejemplos
7. **Arquitectura Event-Driven**
    - 7.1 Arquitectura reactiva y eventos asincrónicos
    - 7.2 Casos de éxito
    - 7.3 Cómo implementar un sistema Event-Driven
8. **Arquitectura Serverless**
    - 8.1 Concepto y modelo de funcionamiento
    - 8.2 Beneficios en términos de escalabilidad y costos
    - 8.3 Limitaciones y cuándo no usar Serverless
9. **Arquitectura CQRS**
    - 9.1 Separación de comandos y consultas
    - 9.2 Escalabilidad con CQRS
    - 9.3 Uso de CQRS junto con Event Sourcing

V.

1. Introducción

¿Qué es una Arquitectura de Software?

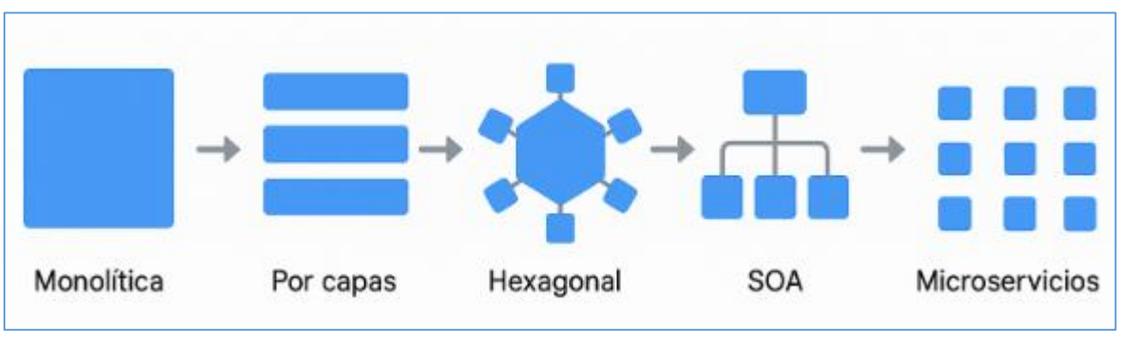
La arquitectura de software es una disciplina clave en el diseño y desarrollo de sistemas de software. Define la estructura del sistema, cómo se descompone en módulos o componentes, y cómo estos se relacionan entre sí. Tomar decisiones arquitectónicas tempranas en el ciclo de desarrollo del software es crucial, ya que pueden afectar la escalabilidad, mantenibilidad, seguridad y flexibilidad del sistema.

Hoy en día, existen diversas arquitecturas que buscan satisfacer diferentes necesidades. Algunas priorizan la simplicidad y velocidad de desarrollo, mientras que otras están orientadas a la escalabilidad y la modularidad. Desde los enfoques más tradicionales, como las arquitecturas monolíticas, hasta las más modernas como microservicios o serverless, cada una tiene sus ventajas y limitaciones.

En este conjunto de clases, exploraremos estas arquitecturas en profundidad, entendiendo no solo cómo funcionan, sino también cuándo y por qué utilizarlas en el diseño de soluciones de software.

Importancia de las Arquitecturas de Software

Las arquitecturas de software establecen las bases sobre las cuales se desarrollan y evolucionan las aplicaciones. Sin una arquitectura adecuada, las aplicaciones pueden enfrentar problemas como dificultad para escalar, tiempos de respuesta elevados o un mantenimiento costoso. Una buena arquitectura no solo se enfoca en resolver problemas actuales, sino que también permite a los sistemas adaptarse a futuros cambios y requisitos.



2. Arquitectura Monolítica

¿Qué es?

La arquitectura monolítica es una de las formas más tradicionales de organizar una aplicación. En este enfoque, toda la aplicación se encuentra dentro de una única unidad de código que maneja todas las responsabilidades, como la interfaz de usuario, la lógica de negocio y el acceso a datos.

Ventajas y Desventajas

Ventajas:

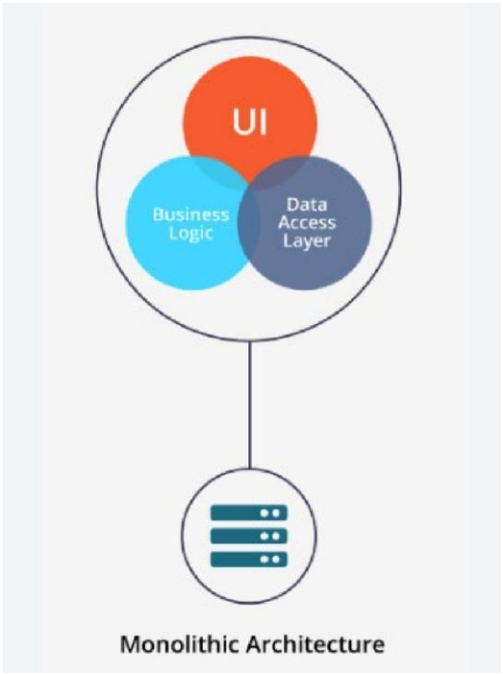
- **Facilidad de desarrollo inicial**: Al estar todo en un solo código base, el proceso de desarrollo es más simple y rápido para proyectos pequeños o medianos.
- **Despliegue sencillo**: Solo se necesita desplegar una unidad de software en el servidor.

Desventajas:

- **Escalabilidad limitada**: A medida que la aplicación crece, se vuelve más difícil de escalar, ya que cada parte está interconectada.
- **Riesgo de cambios inadvertidos**: Cualquier cambio en una parte del sistema puede tener impactos inesperados en otras áreas.
- **Dificultad para introducir nuevas tecnologías**: Es complicado modernizar una parte del sistema sin afectar al resto.

Ejemplos de Uso

Muchas aplicaciones web de pequeña y mediana escala siguen utilizando una arquitectura monolítica. Un ejemplo típico son las aplicaciones creadas con frameworks como Django o Laravel, donde el código está organizado en módulos, pero sigue siendo parte de un único sistema.



---

3. Arquitectura de Capas

¿Qué es?

La arquitectura de capas divide una aplicación en segmentos o "capas", cada una con una responsabilidad claramente definida. Estas capas suelen incluir la capa de presentación, la capa de lógica de negocio y la capa de acceso a datos. La capa superior solo interactúa con la capa inmediata inferior, creando un flujo de comunicación estructurado.

Componentes de la Arquitectura de Capas

1. **Capa de Presentación**: Responsable de la interfaz de usuario y la interacción con el cliente.
2. **Capa de Lógica de Negocio**: Encargada de la lógica central de la aplicación, es decir, las reglas que determinan cómo funciona el sistema.
3. **Capa de Acceso a Datos**: Gestiona las operaciones de lectura y escritura en las bases de datos.

Beneficios y Limitaciones

Beneficios:

- **Separación clara de responsabilidades**: Facilita la comprensión del sistema y permite que los desarrolladores se especialicen en una capa.

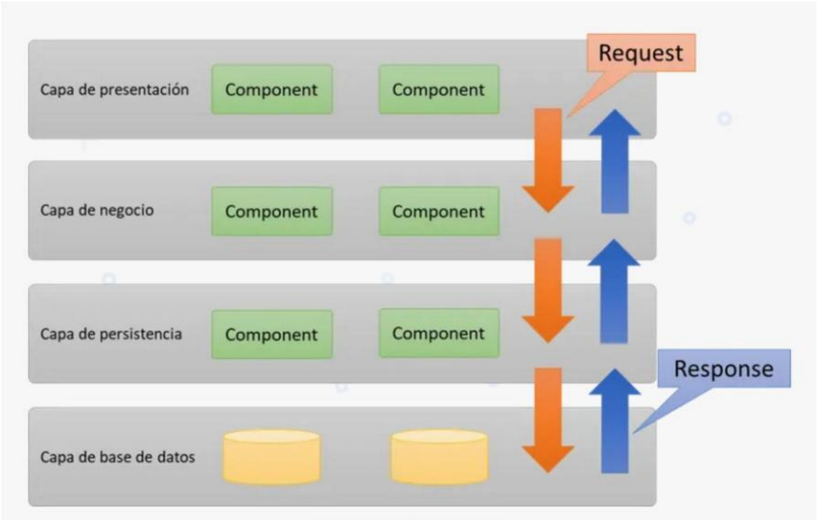
- **Facilidad para realizar pruebas y mantenimiento**: Al tener capas bien definidas, es más fácil aislar y corregir errores.

Limitaciones:

- **Puede volverse rígida**: A medida que crece la aplicación, la interacción entre capas puede volverse un cuello de botella en términos de rendimiento.

Casos de Estudio

Un ejemplo típico de arquitectura en capas es una aplicación web empresarial, donde la capa de presentación está construida con Angular o React, la capa de lógica de negocio con Node.js o .NET, y la capa de acceso a datos maneja interacciones con bases de datos como PostgreSQL o MongoDB.



Comunicación entre capas.

---

4. Arquitectura Hexagonal (Ports and Adapters)

¿Qué es?

La arquitectura hexagonal, también conocida como Ports and Adapters, es un enfoque que busca desacoplar la lógica de negocio de las dependencias externas. Esto incluye bases de datos, APIs, servicios externos, interfaces de usuario, entre otros. La idea principal es que la lógica de negocio debe estar en el "núcleo" de la aplicación, y cualquier interacción con el exterior se debe hacer a través de puertos (interfaces) y adaptadores.

Estructura de una Arquitectura Hexagonal

La arquitectura hexagonal se basa en la idea de tener un núcleo central con la lógica de negocio y, alrededor de este núcleo, los puertos y adaptadores que se encargan de interactuar con el mundo exterior.

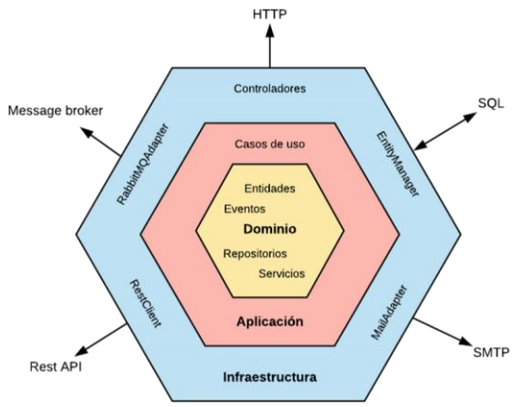
- **Puertos**: Son interfaces que definen cómo la lógica de negocio interactúa con el exterior.
- **Adaptadores**: Son implementaciones de estas interfaces que permiten que la lógica de negocio se comunique con las bases de datos, APIs u otros sistemas.

Patrón de Diseño Aplicable

Esta arquitectura es un ejemplo de la aplicación del **Principio de Inversión de Dependencia** del paradigma SOLID, en el que las dependencias se invierten para que la lógica de negocio no dependa de los detalles de implementación.

Casos de Uso y Ejemplos Prácticos

Este tipo de arquitectura es útil cuando se desea crear una aplicación que puede ser probada de manera aislada, sin depender de componentes externos como bases de datos o interfaces de usuario. Es comúnmente utilizada en aplicaciones de larga duración o sistemas donde se espera que las dependencias externas cambien con el tiempo.



---

5. Arquitectura Basada en Servicios (SOA)

¿Qué es?

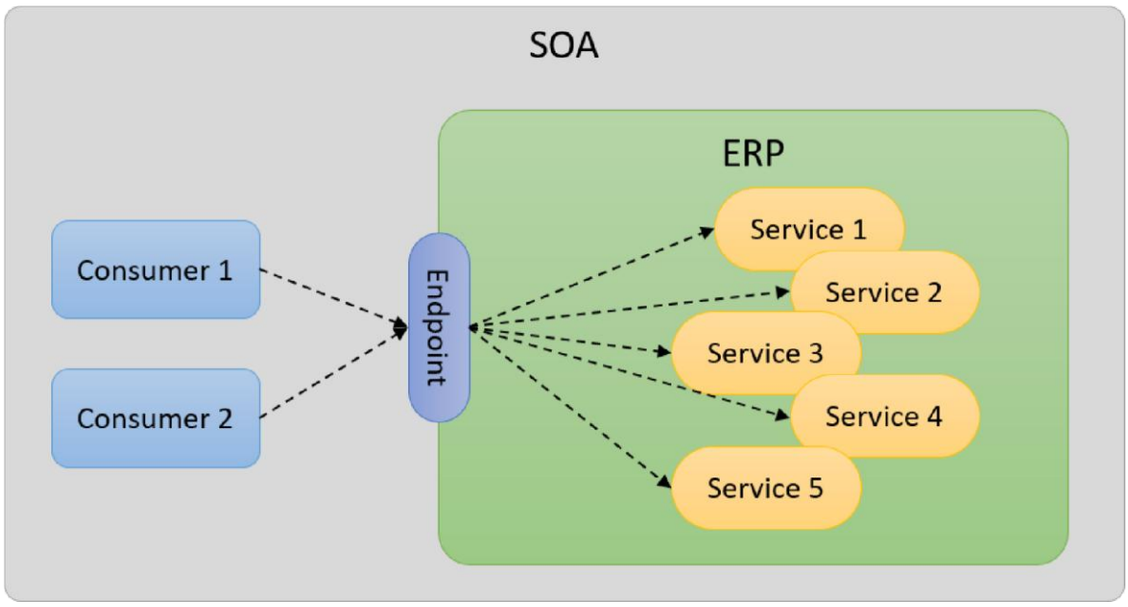
La arquitectura orientada a servicios (SOA) es un estilo de arquitectura de software en el que los servicios son componentes de software que realizan una función específica y pueden interactuar entre sí mediante interfaces de red. SOA se basa en la reutilización de estos servicios y la flexibilidad para desarrollar sistemas grandes y escalables.

Comparativa entre SOA y Microservicios

Si bien SOA y microservicios comparten algunas características, como la división del sistema en componentes autónomos, SOA tiende a tener un enfoque más centrado en la reutilización y la comunicación mediante un bus de servicios, mientras que los microservicios se enfocan más en la independencia total de cada componente.

Escenarios de Aplicación

SOA es ideal para grandes organizaciones que tienen muchos sistemas heredados y necesitan integrarlos de manera eficiente. También es útil cuando se necesita desarrollar aplicaciones escalables en las que los servicios pueden ser reutilizados por diferentes sistemas.



---

6. Microservicios

La arquitectura de microservicios es una evolución de la arquitectura basada en servicios (SOA) que divide una aplicación en servicios más pequeños, completamente autónomos y desplegables por separado. Cada microservicio se encarga de una función o dominio de negocio específico y puede ser desarrollado y mantenido de manera independiente.

6.1 Arquitectura de Microservicios: Independencia y Modularidad

Uno de los principios fundamentales de los microservicios es la **independencia**. Cada microservicio puede ser desarrollado, desplegado y escalado de manera autónoma sin afectar al resto de los servicios. Esto también significa que los equipos de desarrollo pueden trabajar de manera independiente en diferentes microservicios.

La **modularidad** es otro beneficio clave. Dado que cada microservicio se centra en un solo dominio de negocio, se puede gestionar de manera más eficiente en términos de mantenibilidad y escalabilidad.

6.2 Desafíos y Patrones Comunes

Desafíos:

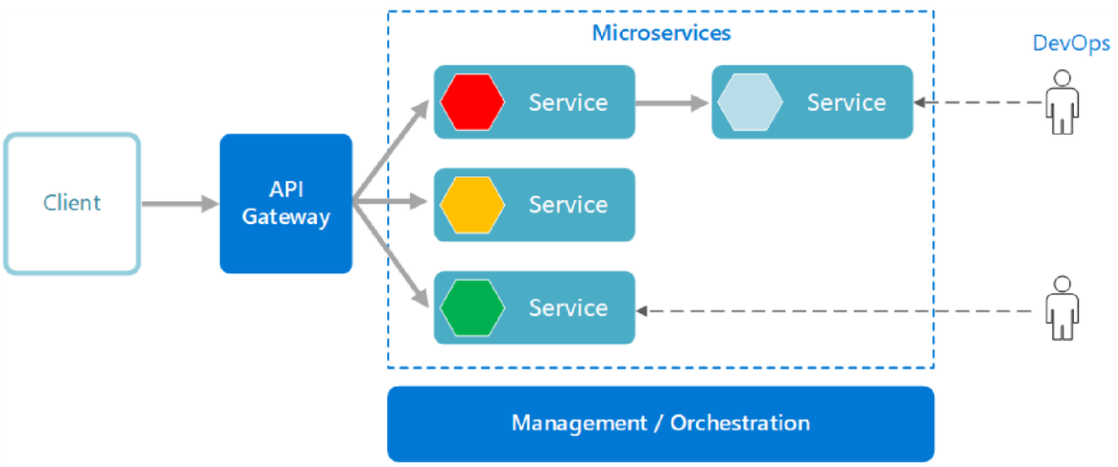
- La comunicación entre los microservicios puede volverse compleja.
- Puede haber duplicación de datos entre microservicios.
- La gestión de transacciones distribuidas es más complicada.

Patrones Comunes:

- **API Gateway**: Un punto de entrada único para todas las solicitudes a los microservicios.
- **Circuit Breaker**: Un patrón para manejar fallos de servicio de manera resiliente.

6.3 Herramientas Populares y Ejemplos

Ejemplos de herramientas populares para implementar microservicios incluyen **Docker** para la contenedorización y **Kubernetes** para la orquestación. Algunas compañías como Netflix y Amazon utilizan microservicios para escalar sus operaciones globales.



7. Arquitectura Event-Driven

En la arquitectura dirigida por eventos (**Event-Driven Architecture**), las acciones dentro de un sistema generan eventos que son capturados por otros componentes, los cuales procesan y reaccionan a dichos eventos de forma asincrónica.

7.1 Arquitectura Reactiva y Eventos Asincrónicos

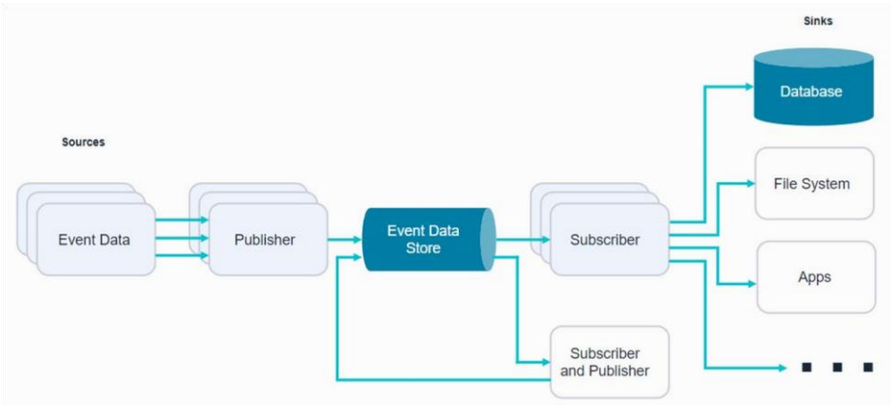
Una arquitectura reactiva se basa en el concepto de **eventos asincrónicos**. Cuando ocurre un evento (por ejemplo, un cambio en el estado de un pedido), ese evento es emitido y otros componentes del sistema reaccionan a él.

7.2 Casos de Éxito

Este tipo de arquitectura se ha utilizado con éxito en sistemas como **Facebook**, donde las actualizaciones en tiempo real son cruciales. Otras plataformas como **Twitter** y **Uber** también utilizan arquitecturas basadas en eventos para procesar grandes cantidades de datos en tiempo real.

7.3 Cómo Implementar un Sistema Event-Driven

Para implementar un sistema Event-Driven se puede utilizar herramientas como **Apache Kafka** o **RabbitMQ** para manejar la cola de eventos y procesarlos de forma asincrónica.



8. Arquitectura Serverless

La arquitectura serverless es un modelo de ejecución en la nube donde el proveedor de servicios se encarga de gestionar la infraestructura, permitiendo que los desarrolladores solo se concentren en escribir código.

8.1 Concepto y Modelo de Funcionamiento

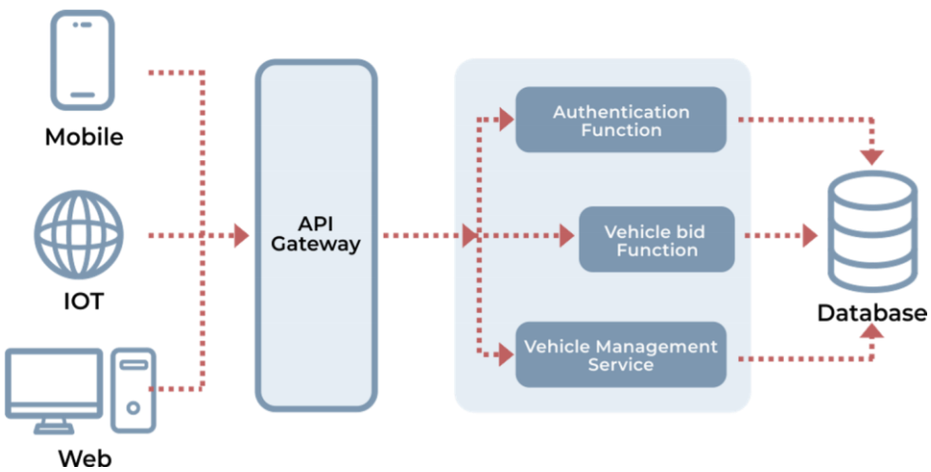
En una arquitectura **serverless**, las funciones se ejecutan en la nube en respuesta a eventos (por ejemplo, una solicitud HTTP o una actualización en una base de datos). Los desarrolladores no gestionan servidores; en su lugar, pagan solo por el tiempo que se ejecuta la función. *V.*

8.2 Beneficios en Términos de Escalabilidad y Costos

- **Escalabilidad automática**: El proveedor de servicios ajusta automáticamente los recursos en función de la demanda.
- **Costos optimizados**: Los costos se basan en el uso real, lo que lo hace ideal para aplicaciones con cargas de trabajo variables.

8.3 Limitaciones y Cuándo No Usar Serverless

- **Latencia**: Las funciones serverless pueden tener una latencia inicial al activarse.
- **No es ideal para cargas de trabajo a largo plazo o constantes**: Serverless está diseñado para tareas de corta duración, y puede no ser la mejor opción para procesos que requieren largos tiempos de ejecución.



---

9. Arquitectura CQRS (Command Query Responsibility Segregation)

La arquitectura **CQRS** separa las responsabilidades de manejo de consultas y comandos. En este modelo, las operaciones de lectura (consultas) y escritura (comandos) se manejan de manera separada, lo que permite optimizar cada una según sus necesidades específicas.

9.1 Separación de Comandos y Consultas

Al separar las operaciones de **lectura** y **escritura**, se puede optimizar el rendimiento para cada operación. Las operaciones de escritura se pueden manejar con transacciones y consistencia fuerte, mientras que las consultas pueden ser optimizadas para el rendimiento.

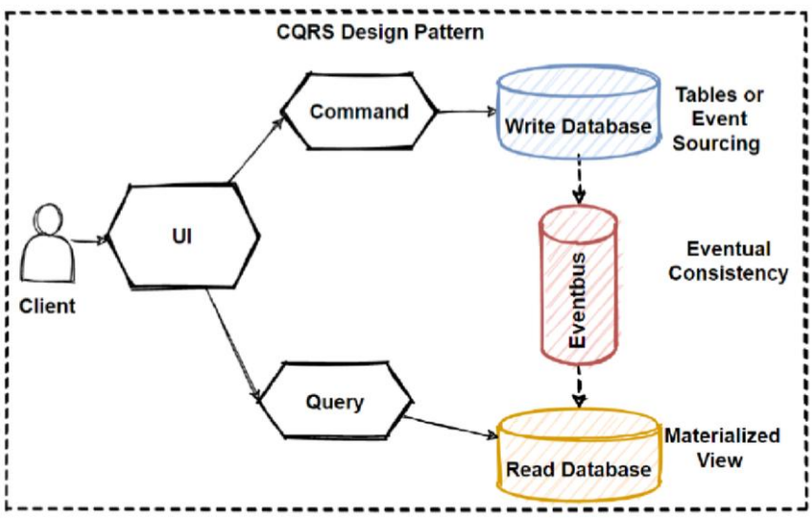
9.2 Escalabilidad con CQRS

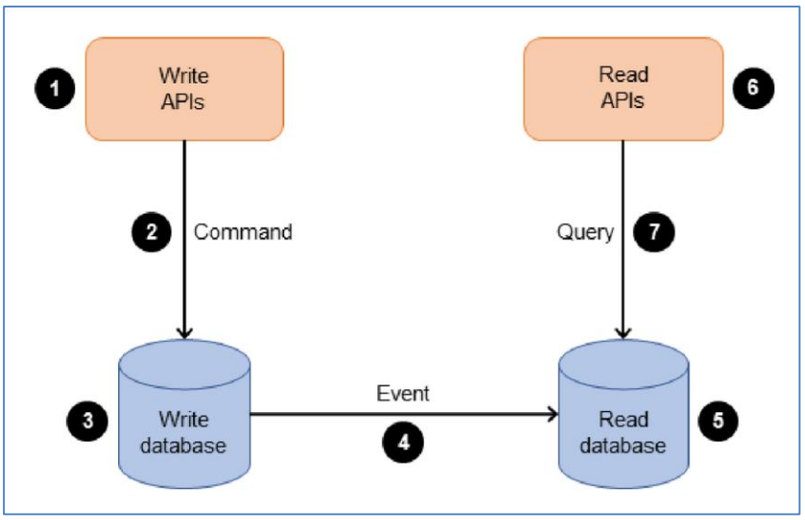
Este modelo permite escalar de manera independiente las operaciones de lectura y escritura, lo cual

es útil en sistemas con un alto volumen de consultas y menor volumen de actualizaciones. *V.*

9.3 Uso de CQRS Junto con Event Sourcing

Cuando se combina con **Event Sourcing**, CQRS permite reconstruir el estado del sistema en cualquier momento, basándose en una secuencia de eventos pasados. Esto es particularmente útil en sistemas financieros o en sistemas donde es importante conservar un historial detallado de las operaciones.





V. 1